

Testing

Testing

UNIT TESTING



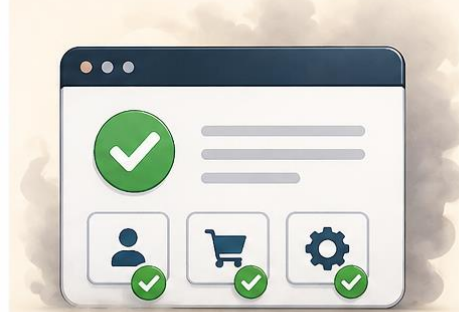
Test individual components or functions in isolation

INTEGRATION TESTING



Test interactions between multiple components or modules

SMOKE TESTING



Quickly verify that the build is stable and critical functions work

REGRESSION TESTING



Ensure existing features still work after changes or bug fixes

PERFORMANCE TESTING



Evaluate speed, stability, scalability and resource usage under load

Testing

1. Unit Testing

- Unit testing focuses on verifying **individual components** or functions of the software in isolation. It ensures that each unit performs as expected independently. Typically, developers write and run these tests during the **development** phase

2. Integration Testing

- Integration testing assesses the **interaction between different modules** or services within the application. It ensures that combined components function together correctly, identifying issues in interfaces and data flow between modules

3. Smoke Testing

- Smoke testing involves a preliminary check to ensure the **basic functionalities** of the application work correctly. It's often referred to as a "sanity check" before proceeding to more rigorous testing.

4. Regression Testing

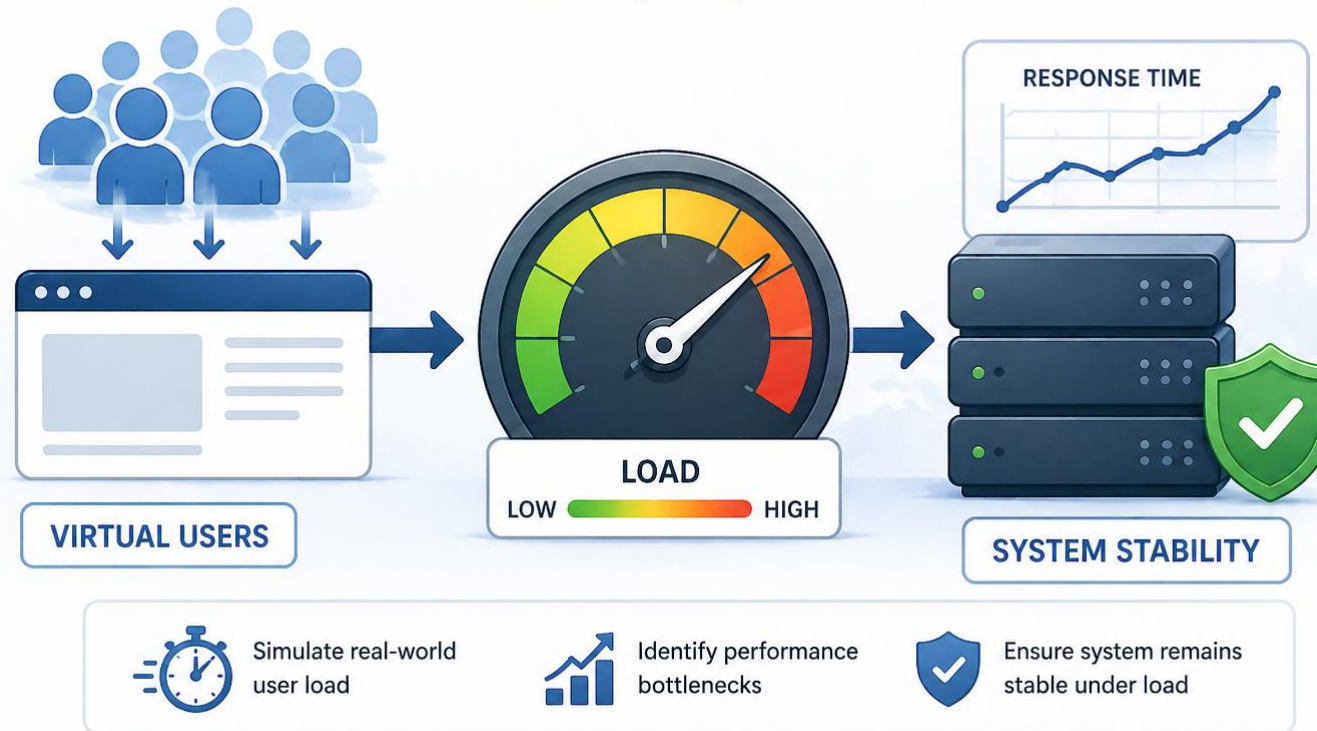
- Regression testing ensures that recent code changes **haven't adversely affected existing functionalities**. It involves re-running previous test cases to confirm that the software continues to perform as expected.

5. Performance Testing

- Performance testing evaluates the software's responsiveness, stability, and scalability under various load conditions. It includes:
 - **Load Testing:** Assesses the system's behaviour under expected user loads.
 - **Stress Testing:** Determines the system's robustness by testing beyond normal operational capacity. To Find the system's *breaking point*.
 - **Endurance Testing:** Checks the system's stability over an extended period of sustained load.

Load Testing

Test system behavior under expected and peak load to ensure stability and performance.



As an AI Engineer, how can I benefit from load testing?

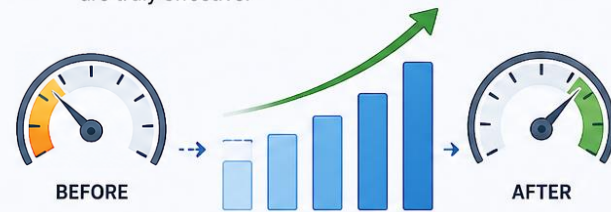
Load Testing

When should I use load tests?

Validate system performance and reliability under real-world load

1 VERIFY PERFORMANCE IMPROVEMENTS

When making performance improvements, we can check if the improvements are truly effective.



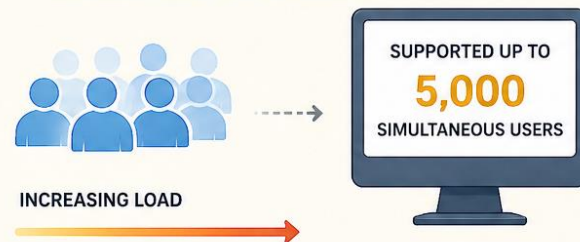
2 COMPARE TECHNOLOGY OPTIONS

Use load tests to compare multiple approaches to the same problem and determine which works best.



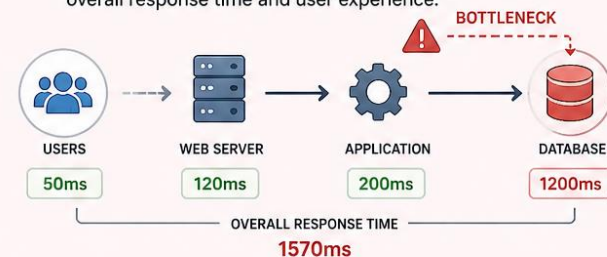
3 ESTIMATE SUPPORTED USERS

Estimate how many simultaneous users are supported by the given system.



4 FIND BOTTLENECKS

Find bottlenecks, that is, components of the system that will take too long to respond and affect the overall response time and user experience.

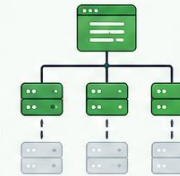


Load Testing – Locust



Test scenarios can be written in Python

Define users, tasks, and behaviors using simple and powerful Python code.



Distributed and scalable

Generate millions of users by distributing load across multiple machines.



Web-based UI

Easy to use, intuitive web interface to configure and monitor your load tests.

LOCUST

A modern load testing tool built for scale.



Any system can be tested

Test web applications, APIs, microservices, databases, and more.



Run from CLI or Web UI

Locust tests can be run from command line or using its web-based UI.



Real-time insights and exportable results

Throughput, response times and errors can be viewed in real time and/or exported for later analysis.



Total Requests

 **25,842**
req/s

Failures

 **129**
(0.50%)

Average Response Time

 **234**
ms

RPS

 **512**
req/s

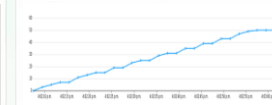
Requests per Second



Response Time (ms)



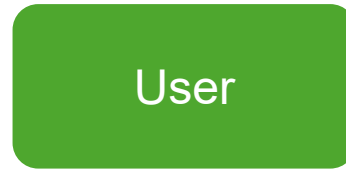
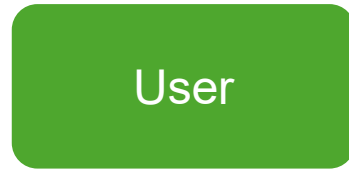
Number of Users



Load Testing – Locust



Behavior 1

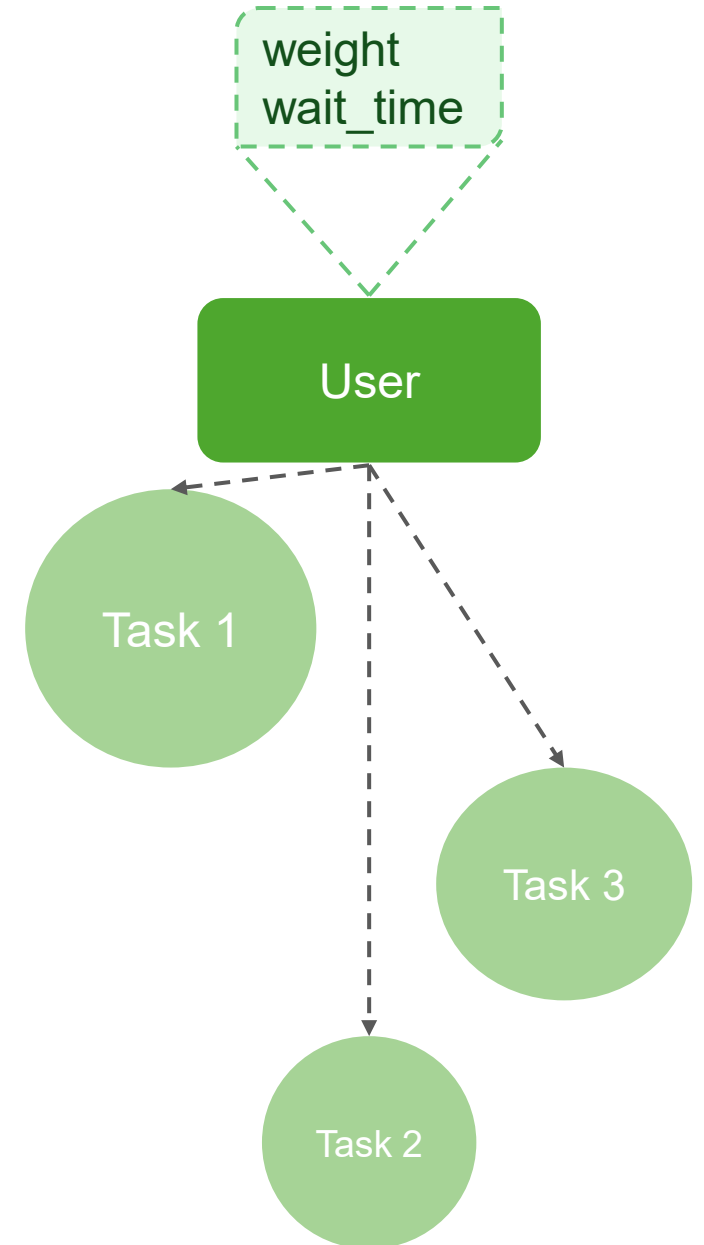


.....

Behavior 2



.....



Load Testing – Locust



A user class represents one type of user/scenario for your system. When you do a test run you specify the number of concurrent users you want to simulate and Locust will create an instance per user. Each of these users will start running within their own green gevent thread.

User classes:

- `HttpUser`
 - Gives each user a client attribute
- `FastHttpUser`
 - Uses `geventhttpclient` instead of `python-requests` as its underlying client.
 - It uses considerably less CPU on the load generator, and should work as a simple drop-in-replacement in most cases.

Load Testing – Locust

The `self.client` attribute makes it possible to make HTTP calls that will be logged by Locust. It contains methods for all HTTP methods: `get`, `post`, `put`, ...

```
from locust import HttpUser

class MyUser(HttpUser):

    @task
    def index(self):
        self.client.get("/")

    @task
    def about(self):
        self.client.get("/about/")
```

Load Testing – Locust

You can add any attributes you like to the user classes/instances, but there are some that have special meaning to Locust:

- **weight**
 - If you wish to simulate more users of a certain type than another, you can set a weight attribute on those classes.
 - If more than one user class exists in the file, and no user classes are specified on the command line, Locust will spawn *an equal number* of each of the user classes.
- **fixed_count**
 - If you set the fixed_count attribute, the weight attribute will be ignored and only that exact number users will be spawned.
 - These users are spawned before any regular, weighted ones.

```
class WebUser(User):  
    weight = 3  
    ...  
  
class MobileUser(User):  
    weight = 1  
    ...
```

```
class AdminUser(User):  
    wait_time = constant(600)  
    fixed_count = 1  
  
    @task  
    def restart_app(self):  
        ...  
  
class WebUser(User):  
    ...
```

Load Testing – Locust



You can add any attributes you like to the user classes/instances, but there are some that have special meaning to Locust:

- **wait_time**

- If no *wait_time* is specified, the next task will be executed as soon as one finishes.
- *constant*: for a fixed amount of time
- *between*: for a random time between a min and max value
- *constant_pacing*: for an adaptive time that ensures the task runs (at most) once every X seconds (it is the mathematical inverse of *constant_throughput*).
- *constant_throughput*: for an adaptive time that ensures the task runs (at most) X times per second.

```
from locust import HttpUser, task, between

class MyUser(HttpUser):
    wait_time = between(5, 15)

    @task(4)
    def index(self):
        self.client.get("/")

    @task(1)
    def about(self):
        self.client.get("/about/")
```

```
class AdminUser(User):
    wait_time = constant(600)
    fixed_count = 1

    @task
    def restart_app(self):
        ...

class WebUser(User):
    ...
```

```
class MyUser(User):
    wait_time = constant_pacing(10)
    @task
    def my_task(self):
        time.sleep(random.random())
```



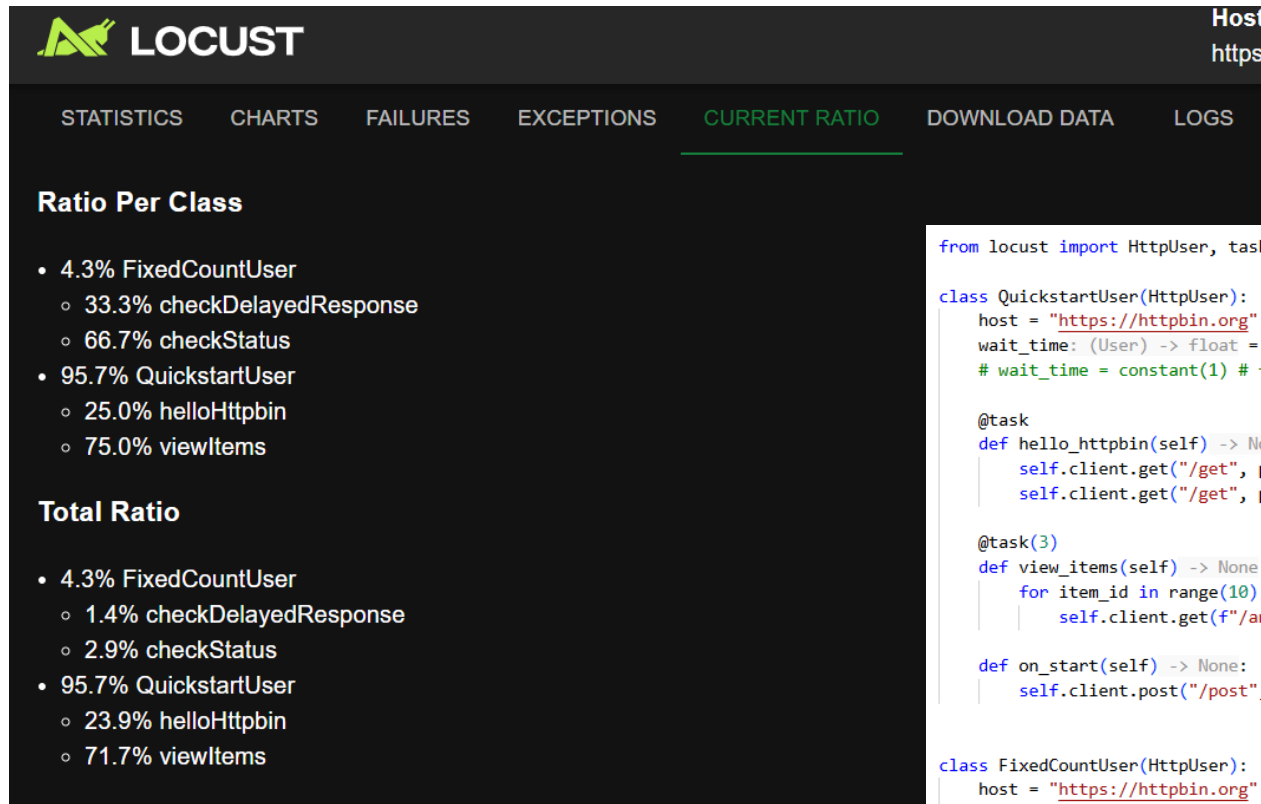
Let's Code IT

Load Testing – Locust

Installation

```
> pip install locust
```

Load Testing – Locust



```
from locust import HttpUser, task, between, constant

class QuickstartUser(HttpUser):
    host = "https://httpbin.org"
    wait_time: (User) -> float = between(1, 5) # for a random time between a min and max value in seconds
    # wait_time = constant(1) # for a fixed amount of time

    @task
    def hello_httpbin(self) -> None:
        self.client.get("/get", params={"source": "quickstart", "route": "hello"})
        self.client.get("/get", params={"source": "quickstart", "route": "world"})

    @task(3)
    def view_items(self) -> None:
        for item_id in range(10):
            self.client.get(f"/anything/item/{item_id}", name="/anything")

    def on_start(self) -> None:
        self.client.post("/post", json={"username": "foo", "password": "bar"})

class FixedCountUser(HttpUser):
    host = "https://httpbin.org"
    fixed_count = 2
    wait_time: (User) -> float = constant(1)

    @task(2)
    def check_status(self) -> None:
        self.client.get("/status/200", name="/status")

    @task
    def check_delayed_response(self) -> None:
        self.client.get("/delay/1", name="/delay")
```



Use Case

Load Testing – Locust



Locust Test Report

During: 11/4/2021, 9:03:39 AM - 11/4/2021, 9:11:17 AM

Target Host: <https://django-todo-list-grjv6.ondigitalocean.app>

Script: locustfile.py

[Download the Report](#)

Request Statistics

Method	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	/tasks/	21512	3	1980	36	13291	929	47.0	0.0
POST	/tasks/	4360	1	2002	14	13276	79	9.5	0.0
Aggregated		25872	4	1984	14	13291	786	56.5	0.0

Response Time Statistics

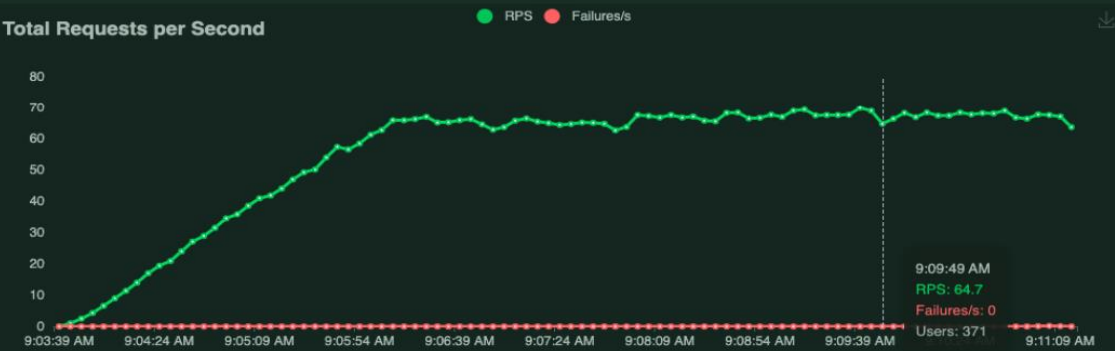
Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	/tasks/	550	740	1200	3600	7100	9800	12000	13000
POST	/tasks/	540	740	1200	3700	7100	9700	12000	13000
Aggregated		550	740	1200	3600	7100	9800	12000	13000

Failures Statistics

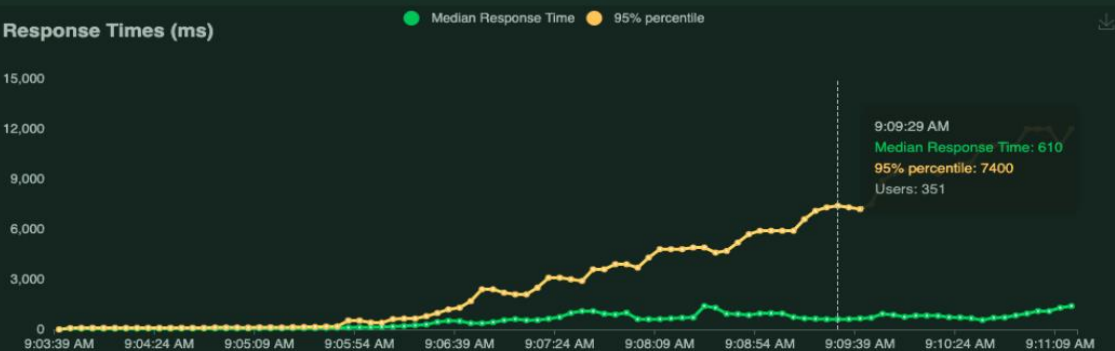
Method	Name	Error	Occurrences
GET	/tasks/	504 Server Error: Gateway Time-out for url: https://django-todo-list-grjv6.ondigitalocean.app/tasks/	3
POST	/tasks/	504 Server Error: Gateway Time-out for url: https://django-todo-list-grjv6.ondigitalocean.app/tasks/	1

Charts

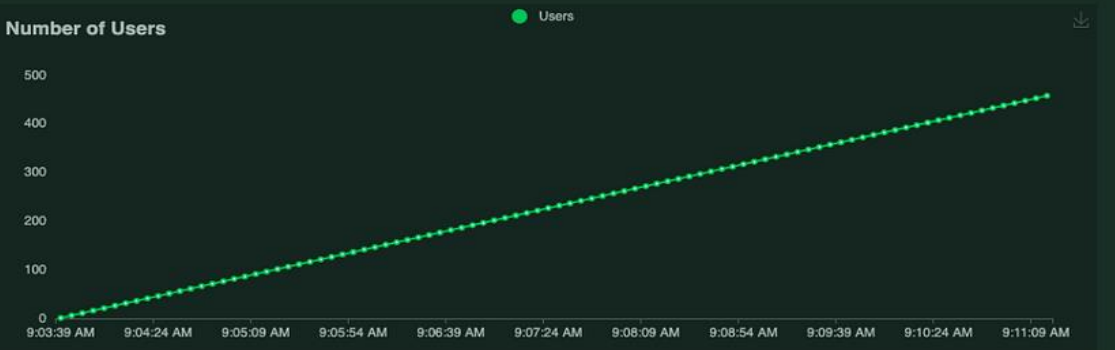
Total Requests per Second



Response Times (ms)



Number of Users



Tasks

Ratio per User class

- 100.0% WebsiteUser
 - 83.3% index
 - 16.7% create

Total ratio

- 100.0% WebsiteUser
 - 83.3% index

Load Testing – Locust

- Although the user number is increasing the RPS is still and there are no failures. Where does all the rest of the users go??

Load Testing – Locust

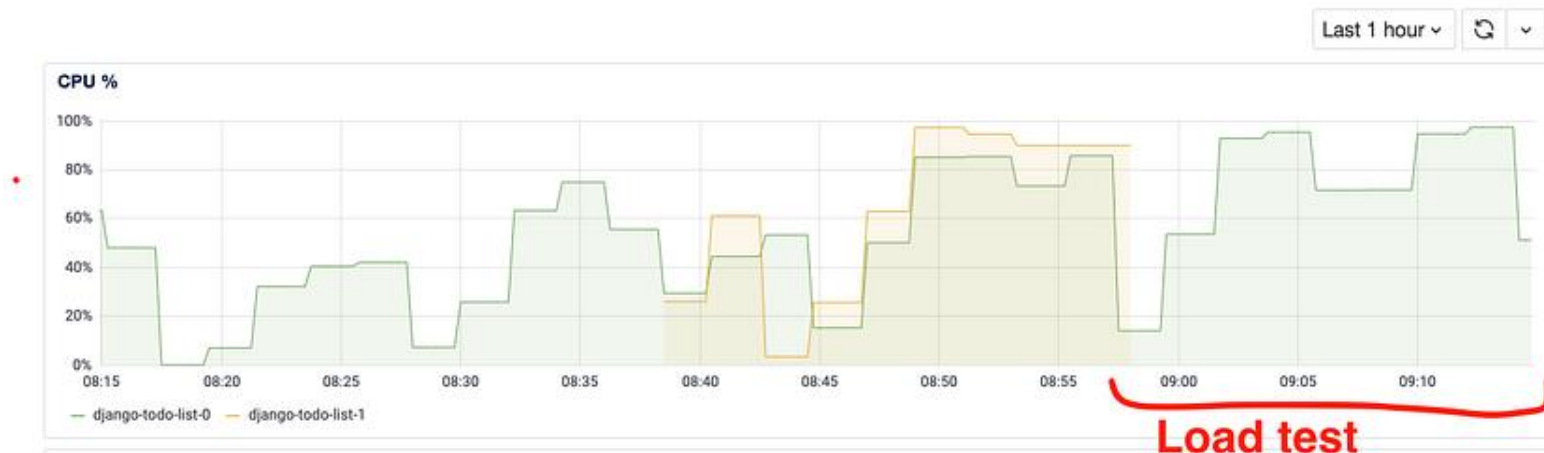


Results and conclusions

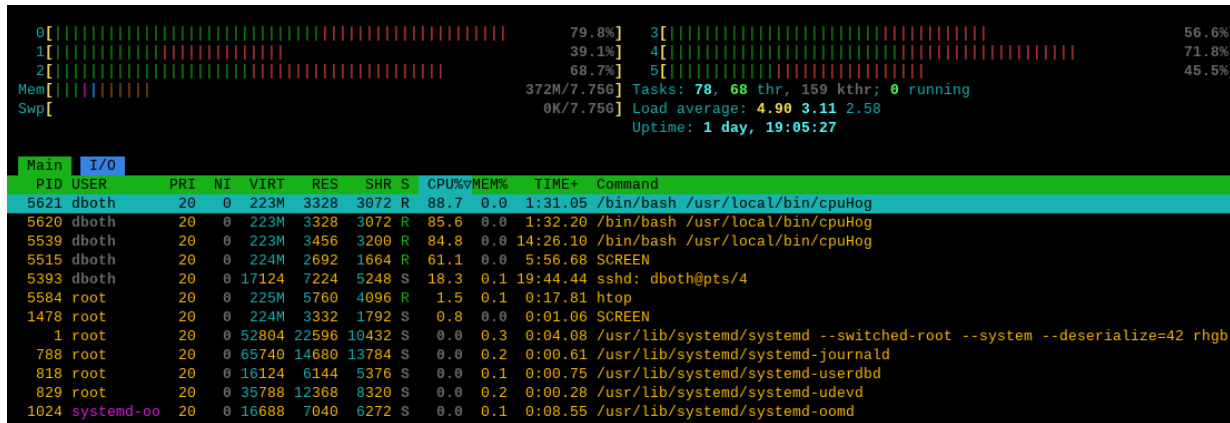
- Until around 160 simultaneous users, we were having around 60 requests per second and the response time was very acceptable for this context (95% of requests finished in less than 400ms)
- After passing 160 simultaneous users, note the number of requests per second remains the same (60rps), but the response time increased a lot. This way, each user started having slower responses.
- In summary, with the available computing resources, we could serve around 160 simultaneous users with 60rps and a response time lower than 400ms. To increase these numbers, we would need more computing resources or make performance adjustments in the code.

Load Testing – Locust

- In our application layer, we see the CPU usage growing until it reached 100%. In a way, it's a good sign that CPU usage is ramping up. This means our application server is well configured and it is not leaving idle resources.
- On the other hand, since our CPU usage is getting to 100%, this also means we need more computing resources.



Commands to use: Htop & nvidia-smi



```
C:\Users\RTX>nvidia-smi
Tue Apr 28 23:48:37 2026
```

NVIDIA-SMI 580.92				Driver Version: 580.92		CUDA Version: 13.0	
GPU	Name			Driver-Model	Bus-Id	Disp.A	Volatile Uncorr. ECC
Fan	Temp	Perf		Pwr:Usage/Cap		Memory-Usage	GPU-Util Compute M.
							MIG M.
0	NVIDIA T1200 Laptop GPU			WDDM	00000000:01:00.0	On	N/A
N/A	57C	P8		5W / 90W	710MiB / 4096MiB		2% Default
							N/A

Processes :							
GPU	GI ID	CI ID	PID	Type	Process name	GPU Memory Usage	
0	N/A	N/A	3836	C+G	...we\Microsoft.Media.Player.exe	N/A	
0	N/A	N/A	21896	C+G	...1g1gvanyjgm\WhatsApp.Root.exe	N/A	
0	N/A	N/A	23244	C+G	...yb3d8bbwe\Notepad\Notepad.exe	N/A	
0	N/A	N/A	39372	C+G	...ata\Roaming\Zoom\bin\Zoom.exe	N/A	

Load Testing – Locust



One step at a time

- When performing load tests and improving performance, it is important to make **small changes, one at a time**. This way we can verify how much a given change affected the results. In the screenshot below, we can see the exact point where I changed a configuration and the response time improved a lot:
- Combining small changes with monitoring (results and computing resources), we can understand how our system behaves under stress and adjust accordingly.





LAB

Load Testing – Lab

- What you'll do is to load test the local deployed container of churn model of the previous lab:
 - Test your deployment through locust-python script
 - Observe the charts and experiment with different parameter values e.g. number of users
 - Submit your answer to those questions:
 - What's the peak RPS?
 - What other insights did you observe from the graph?

Additional Resources

- [Load Testing Using Locust - Use Case](#)
- [Locust Documentation](#)